

WEEK-2

Session-6: Functional Programming using map()

Shuffle the Array

```
class Solution {  
    public int[] shuffle(int[] nums, int n) {  
        int[] result = new int[2 * n];  
  
        for (int i = 0; i < n; i++) {  
            result[2 * i] = nums[i];  
            result[2 * i + 1] = nums[n + i];  
        }  
  
        return result;  
    }  
}
```

The screenshot shows a test result interface with a dark theme. At the top, there are two tabs: 'Testcase' (checked) and 'Test Result'. Below the tabs, the word 'Accepted' is displayed in green, followed by 'Runtime: 0 ms'. Underneath, there are three buttons: 'Case 1' (checked), 'Case 2' (checked), and 'Case 3' (checked). The 'Input' section shows 'nums =' followed by '[2,5,1,3,4,7]' and 'n =' followed by '3'. The 'Output' section shows '[2,3,5,4,1,7]'. At the bottom, there is a 'Feedback' section with a 'Feedback' button.

Session-7: Functional Programming using filter()

Remove Element

```
class Solution {  
    public int removeElement(int[] nums, int val) {  
        int k = 0;  
  
        for (int i = 0; i < nums.length; i++) {  
            if (nums[i] != val) {  
                nums[k] = nums[i];  
                k++;  
            }  
        }  
  
        return k;  
    }  
}
```

The screenshot displays a test result interface with a dark theme. At the top, there are two tabs: 'Testcase' and 'Test Result', with 'Test Result' being the active tab. Below the tabs, the word 'Accepted' is shown in green, followed by 'Runtime: 0 ms'. Underneath, there are two buttons: 'Case 1' and 'Case 2', both with green checkmarks. The 'Input' section contains two text boxes: the first is labeled 'nums =' and contains '[3,2,2,3]'; the second is labeled 'val =' and contains '3'. The 'Output' section contains a single text box labeled 'Output' with the value '[2,2]'. At the bottom, there is a 'Feedback' section with a 'Feedback' button.

Remove Duplicates from Sorted Array

```
class Solution {  
    public int removeDuplicates(int[] nums) {  
        int k = 1;  
  
        for (int i = 1; i < nums.length; i++) {  
            if (nums[i] != nums[i - 1]) {  
                nums[k] = nums[i];  
                k++;  
            }  
        }  
  
        return k;  
    }  
}
```

☒ Testcase | [> Test Result](#)

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2

Input
nums =
[1,1,2]

Output
[1,2]

Expected
[1,2]

Session-8: Functional Programming using reduce()

Maximum Subarray

```
class Solution {  
    public int maxSubArray(int[] nums) {  
  
        int currentSum=nums[0];  
        int maxSum=nums[0];  
  
        for(int i=1; i<nums.length; i++){  
  
            currentSum=Math.max(nums[i], currentSum+nums[i]);  
  
            maxSum=Math.max(maxSum, currentSum);  
        }  
        return maxSum;  
    }  
}
```

☒ Testcase | [Test Result](#)

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2 ☒ Case 3

Input

nums =
[-2,1,-3,4,-1,2,1,-5,4]

Output

6

Expected

6

Find the Highest Altitude

```
class Solution {  
    public int largestAltitude(int[] gain) {  
        int altitude = 0;  
        int highest = 0;  
  
        for (int g : gain) {  
            altitude += g;  
            highest = Math.max(highest, altitude);  
        }  
  
        return highest;  
    }  
}
```

☒ Testcase | [Test Result](#)

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2

Input
gain =
[-5,1,5,0,-7]

Output
1

Expected
1

Session-9: Stream & Pipeline Processing

Group Anagrams

```
import java.util.*;

class Solution {

    public List<List<String>> groupAnagrams(String[] strs) {

        Map<String, List<String>> map = new HashMap<>();

        for (String s : strs) {

            char[] chars = s.toCharArray();

            Arrays.sort(chars);

            String key = new String(chars);

            map.putIfAbsent(key, new ArrayList<>());

            map.get(key).add(s);

        }

        return new ArrayList<>(map.values());

    }

}
```

☒ Testcase | ☒ Test Result

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2 ☒ Case 3

Input

strs =
["eat","tea","tan","ate","nat","bat"]

Output

[["eat","tea","ate"],["bat"],["tan","nat"]]

Expected

[["bat"],["nat","tan"],["ate","eat","tea"]]

Top K Frequent Elements

```
class Solution {  
    public int[] topKFrequent(int[] nums, int k) {  
        java.util.HashMap<Integer, Integer> map = new java.util.HashMap<>();  
  
        for (int n : nums)  
            map.put(n, map.getOrDefault(n, 0) + 1);  
  
        java.util.List<Integer>[] bucket = new java.util.ArrayList[nums.length + 1];  
  
        for (int n : map.keySet()) {  
            int f = map.get(n);  
            if (bucket[f] == null)  
                bucket[f] = new java.util.ArrayList<>();  
            bucket[f].add(n);  
        }  
  
        int[] ans = new int[k];  
        int j = 0;  
  
        for (int i = nums.length; i >= 0 && j < k; i--) {  
            if (bucket[i] != null)  
                for (int n : bucket[i])  
                    ans[j++] = n;  
        }  
  
        return ans;  
    }  
}
```

☒ Testcase | [Test Result](#)

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2 ☒ Case 3

Input

nums =
[1,1,1,2,2,3]

k =
2

Output

[1,2]

Session-10: Stream API & Collection Processing

Java Dequeue

```
import java.util.ArrayDeque;
```

```
import java.util.Deque;
```

```
import java.util.HashMap;
```

```
import java.util.Scanner;
```

```
public class Solution {
```

```
    public static void main(String[] args) {
```

```
        Scanner in = new Scanner(System.in);
```

```
        Deque<Integer> deque = new ArrayDeque<>();
```

```
        HashMap<Integer, Integer> map = new HashMap<>();
```

```
        int n = in.nextInt(); // Total number of elements
```

```
        int m = in.nextInt(); // Size of the contiguous subarray (window)
```

```
        int maxUnique = 0;
```

```

for (int i = 0; i < n; i++) {
    int num = in.nextInt();

    // Add the new element to the tail of the deque and update its frequency
    deque.addLast(num);
    map.put(num, map.getOrDefault(num, 0) + 1);

    // If the window size exceeds 'm', shrink it from the front
    if (deque.size() > m) {
        int first = deque.removeFirst();
        int count = map.get(first);
        if (count == 1) {
            map.remove(first);
        } else {
            map.put(first, count - 1);
        }
    }

    // Update the maximum unique elements once a full window is formed
    if (deque.size() == m) {
        maxUnique = Math.max(maxUnique, map.size());

        // Optimization: If maxUnique equals the window size, we can't get higher
        if (maxUnique == m) {
            break;
        }
    }
}

in.close();
System.out.println(maxUnique);
}
}

```

Congratulations

You solved this challenge. Would you like to challenge your friends?



Next Challenge

✓ Test case 0

✓ Test case 1

✓ Test case 2

✓ Test case 3

✓ Test case 4

✓ Test case 5

✓ Test case 6

Compiler Message

Success

Input (stdin)

```
1 6 3
2 5 3 5 2 3 2
```

[Download](#)

Expected Output

```
1 3
```

[Download](#)